

Conjectura Erdős-Gyárfás

Apostu Cleopatra - grupa 144

În teoria grafurilor, unele dintre cele mai fascinante probleme sunt cele care pot fi explicate foarte simplu, dar care rezistă eforturilor celor mai străluciți matematicieni din lume. Conjectura Erdős-Gyárfás, formulată în anul 1995 de către matematicienii maghiari Paul Erdős și András Gyárfás, este exact o astfel de problemă.

Enunțul problemei

Conjectura se referă la relația dintre gradul minim al unui graf și lungimea ciclurilor pe care acesta le conține. În termeni matematici, enunțul este următorul:

Orice graf simplu G cu gradul minim $\delta(G) \geq 3$ conține cel puțin un ciclu simplu a cărui lungime este o putere a lui 2.

Personalități marcante

Paul Erdős (1913 – 1996)

Paul Erdős este unul dintre cei mai relevanți și excentrici matematicieni ai secolului XX.

- **Stil de viață:** Erdős nu avea o casă fixă. Călătorea dintr-o universitate în alta, cu o singură valiză, bătând la ușa colegilor săi și spunând celebra frază: „Mintea mea este deschisă”. Trăia exclusiv pentru a rezolva probleme de matematică.
- **Recompense financiare:** Obişnuia să ofere premii în bani din propriul buzunar pentru rezolvarea problemelor pe care le propunea (sumele variau de la 10 dolari pentru probleme ușoare, la câteva mii de dolari pentru cele foarte grele).

András Gyárfás (n. 1945)

András Gyárfás este un cercetător maghiar contemporan, activ la Institutul de Calcul și Automatizări din cadrul Academiei Maghiare de Științe.

- Este recunoscut în special pentru cercetările sale legate de colorarea grafurilor și teoria Ramsey.
- O altă problemă celebră care îi poartă numele este **Conjectura Gyárfás-Sumner**, care explorează proprietățile grafurilor care nu conțin anumite subgrafuri specifice (precum arborii induși).

Abordări și progrese în rezolvare

1. Verificări computaționale

O modalitate de a ataca o conjectură este de a căuta un contraexemplu (un graf care respectă regula $\delta(G) \geq 3$, dar nu are niciun ciclu de lungime 2^k).

Matematicienii au folosit algoritmi de căutare exhaustivă pentru a verifica grafurile mici:

- **Gordon Royle și Brendan McKay (anii '90 - 2000):** Folosind programe extrem de optimizate pentru generarea grafurilor au verificat conjectura pentru grafuri de dimensiuni mici.
- **Rezultatul:** S-a demonstrat cu ajutorul calculatorului că conjectura este adevărată pentru toate grafurile care au până la 10 vârfuri. Pentru grafuri mai mari de 10 vârfuri, numărul posibilităților explodează exponențial, făcând verificarea prin „forță brută” imposibilă chiar și cu supercomputerele moderne.

2. Rezultate teoretice pe clase specifice de grafuri

A. Grafuri planare (Planar graphs)

Un graf planar este un graf care poate fi desenat pe o foaie de hârtie fără ca muchiile sale să se intersecteze.

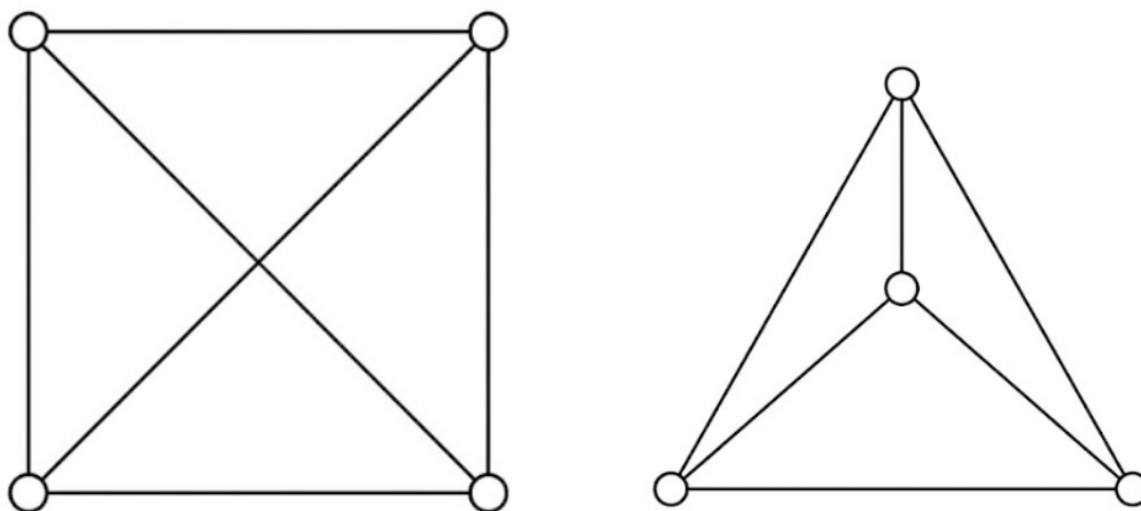


Figura 1: Exemple de grafuri planare.

În 2007, matematicienii Stephenson, Vumar și alții au făcut progrese, arătând că pentru anumite subclase de grafuri planare, conjectura este adevărată.

Ulterior, o serie de teoreme au relaxat condițiile. Deși nu există o demonstrație simplă pentru absolut toate grafurile planare, s-a arătat că dacă un graf planar îndeplinește condiția $\delta(G) \geq 3$ și anumite condiții legate de fețele grafului (cum ar fi grafurile care exclud anumite subgrafuri specifice), atunci are un ciclu de putere a lui 2.

B. Grafuri cubice speciale (Cubic graphs)

Un graf cubic (sau 3-regulat) este un graf în care fiecare vârf are exact gradul 3. Acesta este cazul limită al conjecturii (cel mai mic grad permis).

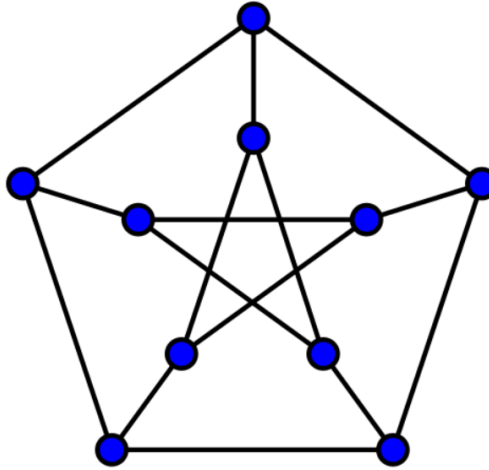


Figura 2: Graful lui Petersen - graf cubic

Matematicienii au arătat interes masiv pentru această clasă, deoarece dacă există un contraexemplu, e foarte probabil să fie găsit aici.

Rezultat notabil (2001): Matematicienii Markström și colegii săi au demonstrat conjectura pentru grafuri cubice care sunt lipsite de anumite cicluri scurte (grafuri cu o anumită *girth* - lungimea celui mai scurt ciclu). S-a arătat că pentru ca un graf cubic să fie un contraexemplu, el trebuie să fie extrem de complex și să aibă zeci de vârfuri.

C. Grafuri fără structuri în „stea” (Claw-free graphs)

Un „claw” (gheară) este un subgraf în formă de stea format din 4 vârfuri (un vârf central conectat la 3 frunze care nu sunt conectate între ele, echivalent cu graful bipartit complet $K_{1,3}$).

Shou-Jian Xu și alți matematicieni (în special în anii 2010) au publicat lucrări în care demonstrează că orice graf simplu cu $\delta(G) \geq 3$ care nu conține subgraful „gheară” (*claw-free*) satisface conjectura Erdős-Gyárfás. Acesta este unul dintre cele mai importante rezultate teoretice obținute până în prezent.

3. Variante alternative ale conjecturii

Un alt mod de progres a fost schimbarea întrebării. În loc să ceară cicluri cu lungime exact de forma 2^k , s-au demonstrat teoreme pentru alte seturi de numere:

Variația lui Sudakov și Verstraëte: Ei au arătat un rezultat fascinant legat de lungimea ciclurilor și gradul mediu al grafului. Au demonstrat că dacă gradul mediu al grafului este suficient de mare (chiar dacă nu se referă strict la puteri ale lui 2), setul lungimilor ciclurilor din graf are o densitate remarcabilă, sugerând că, asimptotic, este aproape imposibil să ocolești puterile lui 2 într-un graf foarte dens.

Explorare Computațională

Pentru a testa empiric conjectura pe cazuri specifice și pentru a căuta un potențial contraexemplu, am dezvoltat un script în limbajul Python. Utilizând biblioteca `networkx`, acest cod generează automat grafuri aleatoare și le filtrează pe cele care sunt candidați relevanți (asigurându-se că au gradul minim cel puțin 3, sunt neplanare și conțin structuri de tip „gheară”). Apoi, algoritmul aplică o căutare exhaustivă (*Depth-First Search*) pentru a identifica dacă există cicluri de lungime 2^k . Pentru eficiență, scriptul memorează grafurile deja evaluate (folosind amprente/hash-uri) pentru a nu repeta testele pe structuri identice.

```

import networkx as nx
import itertools
import os

NUMAR_GRAFURI = 100
NODURI = 41
FISIER_ISTORIC = "istoric_grafuri.txt"

def has_claw(G):
    """Verifica daca graful contine un subgraf 'gheara' (K1,3)."""
    for node in G.nodes():
        neighbors = list(G.neighbors(node))
        if len(neighbors) >= 3:
            for n1, n2, n3 in itertools.combinations(neighbors, 3):
                if not G.has_edge(n1, n2) and not G.has_edge(n2, n3) and not G.has_edge(n1, n3):
                    return True
    return False

def find_cycle_of_length(G, target_length):
    """Cauta un ciclu simplu de lungime target_length."""
    def dfs(current, start, path, visited):
        if len(path) == target_length:
            if G.has_edge(current, start):
                return path
            return None

        for neighbor in G.neighbors(current):
            if neighbor not in visited:
                visited.add(neighbor)
                path.append(neighbor)
                result = dfs(neighbor, start, path, visited)
                if result: return result
                path.pop()
                visited.remove(neighbor)
        return None

    for node in G.nodes():
        cycle = dfs(node, node, [node], {node})
        if cycle: return cycle
    return None

def verify_erdos_gyarfas(G):
    """Verifica prezenta unui ciclu de forma 2^k."""
    n = len(G.nodes())
    k = 2
    while (2 ** k) <= n:
        target = 2 ** k
        cycle = find_cycle_of_length(G, target)
        if cycle:
            return target, cycle
        k += 1
    return None, None

def incarca_istoric(ume_fisier):
    """Citeste fisierul si returneaza un set cu amprente grafurilor deja testate."""
    istoric = set()
    if os.path.exists(ume_fisier):
        with open(ume_fisier, "r") as f:

```

```

        for linie in f:
            if " | " in linie:
                hash_graf = linie.split(" | ")[0].strip()
                istoric.add(hash_graf)
    return istoric

def salveaza_graf(nume_fisier, hash_graf, G):
    """Salveaza amprenta si muchiile grafului in fisier."""
    with open(nume_fisier, "a") as f:
        muchii = list(G.edges())
        f.write(f"{hash_graf} | {muchii}\n")

def genereaza_candidat_nou(noduri, istoric_hashuri):
    """Genereaza un graf valid care nu a mai fost testat niciodata."""
    incercari = 0
    while True:
        incercari += 1
        G = nx.erdos_renyi_graph(noduri, p=0.4)

        if min(dict(G.degree()).values(), default=0) < 3:
            continue
        is_planar, _ = nx.check_planarity(G)
        if is_planar:
            continue
        if not has_claw(G):
            continue

        amprenta = nx.weisfeiler_lehman_graph_hash(G)

        if amprenta in istoric_hashuri:
            continue

    return G, amprenta, incercari

if __name__ == "__main__":
    print(f"Cautam {NUMAR_GRAFURI} grafuri noi.")

    istoric = incarca_istoric(FISIER_ISTORIC)
    print(f"Am incarcat {len(istoric)} grafuri testate anterior din baza de
    date.")

    grafuri_testate_acum = 0

    while grafuri_testate_acum < NUMAR_GRAFURI:
        G, amprenta, incercari = genereaza_candidat_nou(NODURI, istoric)
        grafuri_testate_acum += 1

        print(f"\n[{grafuri_testate_acum}/{NUMAR_GRAFURI}] S-a gasit un
        candidat nou dupa {incercari} generari aleatorii.")
        print(f"Amprenta: {amprenta[:8]}... | Muchii: {G.number_of_edges()}")

        lungime, traseu = verify_erdos_gyarfes(G)

        if traseu:
            print(f"DA. S-a gasit un ciclu de lungime {lungime}: {traseu}")
        else:
            print(" NU!")
            with open("CONTRAEXEMPLU_GASIT.txt", "w") as f:
                f.write(f"Muchii: {list(G.edges())}\n")

        istoric.add(amprenta)
        salveaza_graf(FISIER_ISTORIC, amprenta, G)

```

```
print("\nSesiune incheiata. Grafurile testate au fost adaugate in fisierul  
istoric.")
```

```
1  import networkx as nx
2  import itertools
3  import os
4
5
6  NUMAR_GRAFURI = 100
7  NODURI = 20
8  FISIER_ISTORIC = "istoric_grafuri.txt"
9  # -----
10
11 def has_claw(G):
12     """Verifică dacă graful conține un subgraf 'gheară' (K1,3)."""
13     for node in G.nodes():
14         neighbors = list(G.neighbors(node))
15         if len(neighbors) >= 3:
16             for n1, n2, n3 in itertools.combinations(neighbors, 3):
17                 if G.has_edge(n1, n2) and G.has_edge(n1, n3) and G.has_edge(n2, n3):
18                     return True
19     return False
20
21 if __name__ == '__main__':
22     # Generăm grafuri aleatorii și verificăm dacă conțin un subgraf 'gheară'
23     for i in range(1, NUMAR_GRAFURI + 1):
24         G = nx.Graph()
25         G.add_nodes_from(range(20))
26         G.add_edges_from(itertools.combinations(range(20), 2))
27         if has_claw(G):
28             # Salvăm în fișierul istoric
29             with open(FISIER_ISTORIC, 'a') as f:
30                 f.write(f"{i} {G.edges()}\n")
31         else:
32             # Salvăm în fișierul istoric
33             with open(FISIER_ISTORIC, 'a') as f:
34                 f.write(f"{i} {G.edges()}\n")
35     print("\nSesiune incheiata. Grafurile testate au fost adaugate in fisierul istoric.")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\apost\Desktop\LFA\DFA> & C:\Users\apost\AppData\Local\Programs\Python\Python39\python.exe C:\Users\apost\Desktop\LFA\DFA\main.py

DA. S-a găsit un ciclu de lungime 4: [0, 1, 3, 4]

[97/100] S-a găsit un candidat nou după 1 generări aleatorii.
Amprentă: c8bf81f0... | Muchii: 73
DA. S-a găsit un ciclu de lungime 4: [0, 3, 6, 10]

[98/100] S-a găsit un candidat nou după 1 generări aleatorii.
Amprentă: 56dbffa4... | Muchii: 62
DA. S-a găsit un ciclu de lungime 4: [0, 9, 1, 16]

[99/100] S-a găsit un candidat nou după 1 generări aleatorii.
Amprentă: 055e3397... | Muchii: 70
DA. S-a găsit un ciclu de lungime 4: [0, 2, 6, 5]

[100/100] S-a găsit un candidat nou după 1 generări aleatorii.
Amprentă: b531a77d... | Muchii: 72
DA. S-a găsit un ciclu de lungime 4: [0, 1, 3, 5]

Figura 3: Rezultatele experimentelor rulate în terminal (Partea 1).

```
1 import networkx as nx
2 import itertools
3 import os
4
5
6 NUMAR_GRAFURI = 100
7 NODURI = 30
8 FISIER_ISTORIC = "istoric_grafuri.txt"
9 # -----
10
11 def has_claw(G):
12     """Verifică dacă graful conține un subgraf 'gheară' (K1,3)."""
13     for node in G.nodes():
14         neighbors = list(G.neighbors(node))
15         if len(neighbors) >= 3:
16             for n1, n2, n3 in itertools.combinations(neighbors, 3):
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\apost\Desktop\LFA\DFA> & C:\Users\apost\AppData\Local\Programs\Python\Python39\python.exe C:\Users\apost\Desktop\LFA\DFA\main.py

DA. S-a găsit un ciclu de lungime 4: [0, 1, 3, 2]

[95/100] S-a găsit un candidat nou după 1 generări aleatorii.
Amprentă: d8810c63... | Muchii: 181
DA. S-a găsit un ciclu de lungime 4: [0, 1, 2, 19]

[96/100] S-a găsit un candidat nou după 1 generări aleatorii.
Amprentă: a4ceccfd... | Muchii: 165
DA. S-a găsit un ciclu de lungime 4: [0, 7, 1, 16]

[97/100] S-a găsit un candidat nou după 1 generări aleatorii.
Amprentă: 9aca95de... | Muchii: 171
DA. S-a găsit un ciclu de lungime 4: [0, 3, 2, 4]

Figura 4: Rezultatele experimentelor rulate în terminal (Partea 2).

```
1 import networkx as nx
2 import itertools
3 import os
4
5
6 NUMAR_GRAFURI = 100
7 NODURI = 41
8 FISIER_ISTORIC = "istoric_grafuri.txt"
9 # -----
10
11 def has_claw(G):
12     """Verifică dacă graful conține un subgraf 'gheară' (K1,3)."""
13     for node in G.nodes():
14         neighbors = list(G.neighbors(node))
15         if len(neighbors) >= 3:
16             for n1, n2, n3 in itertools.combinations(neighbors, 3):
17                 if G.has_edge(n1, n2) and G.has_edge(n1, n3) and G.has_edge(n2, n3):
18                     return True
19     return False
20
21 def generate_graph():
22     G = nx.Graph()
23     G.add_nodes_from(range(NODURI))
24     edges = []
25     for i in range(NODURI):
26         for j in range(i+1, NODURI):
27             edges.append((i, j))
28     G.add_edges_from(edges)
29     return G
30
31 def find_cycles(G):
32     cycles = []
33     for node in G.nodes():
34         neighbors = list(G.neighbors(node))
35         for n1, n2, n3 in itertools.combinations(neighbors, 3):
36             if G.has_edge(n1, n2) and G.has_edge(n1, n3) and G.has_edge(n2, n3):
37                 cycles.append([node, n1, n2, n3])
38     return cycles
39
40 def main():
41     G = generate_graph()
42     if has_claw(G):
43         print("DA. S-a găsit un ciclu de lungime 4: [0, 3, 4, 13]")
44     else:
45         print("NU. Nu s-a găsit un ciclu de lungime 4.")
46
47     for i in range(100):
48         G = generate_graph()
49         if has_claw(G):
50             print(f"[{i}/100] S-a găsit un candidat nou după 1 generări aleatorii.")
51             print(f"Amprentă: {hash(G)}... | Muchii: {G.number_of_edges()}")
52             cycles = find_cycles(G)
53             if cycles:
54                 print(f"DA. S-a găsit un ciclu de lungime 4: {cycles[0]}")
55             else:
56                 print("NU. Nu s-a găsit un ciclu de lungime 4.")
57         else:
58             print(f"[{i}/100] S-a găsit un candidat nou după 1 generări aleatorii.")
59             print(f"Amprentă: {hash(G)}... | Muchii: {G.number_of_edges()}")
60             cycles = find_cycles(G)
61             if cycles:
62                 print(f"DA. S-a găsit un ciclu de lungime 4: {cycles[0]}")
63             else:
64                 print("NU. Nu s-a găsit un ciclu de lungime 4.")
65
66 if __name__ == "__main__":
67     main()
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

PS C:\Users\apost\Desktop\LFA\DFA> & C:\Users\apost\AppData\Local\Programs\Python\Python310\python.exe C:\Users\apost\Desktop\LFA\DFA\main.py

DA. S-a găsit un ciclu de lungime 4: [0, 3, 4, 13]

[95/100] S-a găsit un candidat nou după 1 generări aleatorii.
Amprentă: c17b1996... | Muchii: 323
DA. S-a găsit un ciclu de lungime 4: [0, 1, 12, 8]

[96/100] S-a găsit un candidat nou după 1 generări aleatorii.
Amprentă: 7c6f5455... | Muchii: 343
DA. S-a găsit un ciclu de lungime 4: [0, 1, 3, 15]

[97/100] S-a găsit un candidat nou după 1 generări aleatorii.
Amprentă: 7a3b8f5f... | Muchii: 333
DA. S-a găsit un ciclu de lungime 4: [0, 1, 4, 3]

[98/100] S-a găsit un candidat nou după 1 generări aleatorii.
Amprentă: b43f98f8... | Muchii: 333
DA. S-a găsit un ciclu de lungime 4: [0, 1, 8, 4]

[99/100] S-a găsit un candidat nou după 1 generări aleatorii.
Amprentă: f0ad2a0c... | Muchii: 322
DA. S-a găsit un ciclu de lungime 4: [0, 1, 3, 5]

Figura 5: Rezultatele experimentelor rulate în terminal (Partea 3).

Intuiție Personală

Primul instinct pe care l-am avut pentru a aborda problema în cazul general a fost utilizarea inducției matematice pe numărul de noduri sau de muchii. Logica mea inițială era că, plecând de la un graf de bază care respectă regula, aş putea elimina un nod pentru a demonstra pasul inductiv. Problema acestei metode s-a dovedit a fi structurală: dacă extragem un nod pentru a apela la ipoteza inductivă, vecinii aceluia nod își pierd câte o conexiune. Astfel, gradele lor scad inevitabil, putând ajunge la 2 sau mai puțin. Acest lucru distruge însăși condiția fundamentală de grad minim a conjecturii și blochează validitatea inducției.

Concluzie

Conjectura Erdős-Gyárfás rămâne unul dintre cele mai frumoase exemple ale contrastului din teoria matematică: o problemă al cărei enunț poate fi înțeles în câteva minute, dar a cărei rezolvare a evadat eforturilor cercetătorilor timp de aproape trei decenii. Am văzut pe parcursul

acestui referat că, deși pare intuitiv ca un graf suficient de conectat (cu $\delta(G) \geq 3$) să ascundă cel puțin un ciclu de lungime 2^k , transformarea acestei intuiții într-o teoremă universală este o sarcină monumentală.

References

<https://en.wikipedia.org/wiki/Erd>

Daniel, Dale; Shauger, Stephen E. (2001), "A result on the Erdős–Gyárfás conjecture in planar graphs", *Proc. 32nd Southeastern Int. Conf. Combinatorics, Graph Theory, and Computing*, pp. 129–139.